Projektowanie aplikacji webowych opartych o mikrousługi

LAB3 — Rejestracja i odnajdywanie usług (Eureka)

Projektowanie aplikacji webowych opartych o mikrousługi

- **Czas:** 120 minut zajęć; konfigurację serwera Eureka i zależności należy przygotować przed zajęciami
- **Technologia:** Python, FastAPI, `py-eureka-client`; Eureka Server uruchamiany z dostarczonego pakietu
- **Forma:** praca indywidualna lub w parach
- **Rezultat:** dwie odrębne usługi zarejestrowane w Eurece oraz wywołanie HTTP odnajdujące usługę po jej nazwie

1. Cel i zakres laboratorium

Na tych zajęciach należy rozwinąć projekt z LAB1 i LAB2 o rejestrację oraz odnajdywanie instancji usług w Eurece. Obie usługi zgłaszają swoją nazwę, adres i port do rejestru. Jedna z nich wykorzystuje informacje z rejestru do wykonania żądania HTTP do drugiej.

Po laboratorium należy umieć:

- odróżnić nazwę usługi od adresu konkretnej jej instancji,
- zarejestrować aplikację w Eurece i wyjaśnić rolę heartbeat,
- odnaleźć usługę po nazwie logicznej zamiast wpisywać jej adres w kodzie,
- wykonać żądanie HTTP do odnalezionej instancji,
- obsłużyć brak usługi lub błąd odpowiedzi zależnej usługi,
- sprawdzić zachowanie integracji testem i udokumentować jej ograniczenia.

Powiązanie z efektami kształcenia

Efekt	Ćwiczone działanie	Dowód wykonania
EK_03 — implementacja funkcjonalności z wykorzystaniem dokumentacji	Integracja dwóch aplikacji z rejestrem i wywołanie przez odnaleziony adres	Kod, testy oraz opis wykorzystanej dokumentacji
EK_04 — projektowanie i komunikacja kilku usług	Uzupełnienie planu z LAB1 o drugą usługę, jej właściciela danych i kontrakt HTTP	Dwie działające usługi, osobne odpowiedzialności i scenariusz komunikacji
EK_01 — rozumienie pojęć	Rozróżnienie rejestru usług, usługi i jej instancji	Wyjaśnienie przepływu rejestracji i odnajdywania
EK_02 — korzystanie z narzędzi	Eureka Server, klient Python, FastAPI i klient HTTP	Powtarzalna konfiguracja i testy
EK_05 — analiza jakości rozwiązania	Analiza błędu odnajdywania lub niedostępności zależnej usługi	Zapisany scenariusz błędu i zaobserwowany rezultat

2. Zachowanie ciągłości projektu

Należy kontynuować repozytorium z LAB1 i LAB2. Nie należy przepisywać działającej usługi ani zastępować jej przykładem z paczki. Przed zmianami należy uruchomić dotychczasowe testy i zapisać działający etap w Git.

Należy zachować:

- temat, granice odpowiedzialności i regułę biznesową z LAB1,
- istniejące API i fabrykę aplikacji `create_app()`,
- trwały zapis i cache z LAB2,
- port pierwszej usługi; dla drugiej należy wybrać inny, wolny port,
- oddzielną własność danych każdej usługi.

Drugą usługę należy wybrać zgodnie z planem zapisanym w LAB1. Powinna obsługiwać własny, mały fragment domeny i udostępniać co najmniej jeden użyteczny endpoint. Nie należy tworzyć drugiej aplikacji wyłącznie z endpointem `/health` ani współdzielić tabel pierwszej usługi. Odczyt danych innej usługi należy realizować przez jej API.

W przypadku pracy w parze należy zmieniać role osoby piszącej i sprawdzającej. W dokumentacji należy wskazać wkład każdej osoby. Projekt semestralny nadal wymaga od każdej osoby implementacji co najmniej dwóch różnych mikrousług.

Praca na zajęciach i po zajęciach

Proponowany podział 120 minut: 10 minut pojęć, 15 minut przykładu, 15 minut decyzji projektowych, 40 minut implementacji, 25 minut weryfikacji, 10 minut odbioru i 5 minut rezerwy.

Punkt kontrolny na koniec zajęć: obie usługi są widoczne w panelu Eureka jako `UP`, a pierwsze żądanie między nimi działa przez nazwę logiczną. Po zajęciach można dokończyć test awarii zależności, testy jednostkowe i dokumentację. Nadal należą one do wymagań oddawanego etapu.

3. Pojęcia potrzebne do wykonania zadania

Rejestr, usługa i instancja

Rejestr usług przechowuje informacje o uruchomionych instancjach: nazwę usługi, adres, port i stan. **Usługa** jest logicznym komponentem, np. `LOCKER-SERVICE`. **Instancja** jest konkretnym uruchomionym procesem tej usługi, np. procesem dostępnym pod `127.0.0.1:8101`.

Rejestracja zgłasza instancję do Eureki. **Heartbeat** to okresowe potwierdzenie jej obecności wysyłane przez klienta. **Odnajdywanie (service discovery)** pozwala wybrać adres instancji na podstawie nazwy logicznej. **Dzierżawa (lease)** określa, jak długo rejestr może uznawać zgłoszenie za ważne bez odnowienia. Wyrejestrowanie podczas poprawnego zamykania procesu różni się od nagłej awarii, przy której klient nie zdąży wysłać takiego zgłoszenia.

Pojęcie	Przykład	Znaczenie
Nazwa usługi	`LOCKER-SERVICE`	Identyfikuje rodzaj usługi, którego szuka klient
Adres instancji	`127.0.0.1:8101`	Wskazuje konkretny proces obsługujący HTTP
Identyfikator zasobu	`KRK-1024`	Wskazuje paczkomat w danych domenowych

Dwie instancje `LOCKER-SERVICE` nadal realizują tę samą usługę. Wymagane w tym etapie dwie różne usługi mają natomiast różne odpowiedzialności domenowe. Można uruchomić je na jednym komputerze; osobne komputery ani maszyny wirtualne nie są wymagane.

Rozmieszczenie procesów i połączeń

Syntax error in text
mermaid version 11.4.1

Linie ciągłe przedstawiają żądania dotyczące danych domenowych, a przerywane — komunikację z rejestrem. W wariancie Docker tylko Eureka działa w kontenerze; obie usługi Python działają lokalnie. Eureka przechowuje adresy usług, a nie dane paczkomatów. Baza i cache własnej pierwszej usługi pozostają jej zasobami z LAB2.

Przebieg odnajdywania i wywołania HTTP

Syntax error in text
mermaid version 11.4.1

Klient `py-eureka-client` pobiera i okresowo odświeża lokalną kopię rejestru, z której wybiera instancję. Nie jest wymagane osobne zapytanie do Eureki przed każdym żądaniem biznesowym. Kopia rejestru jest czymś innym niż cache danych w Redis z LAB2. Rejestracja i heartbeat nie gwarantują, że instancja odpowie na każde żądanie; adres może być chwilowo nieaktualny, a zależna usługa może zwrócić błąd.

Stan `UP` nie jest dowodem poprawności reguły biznesowej, bazy ani Redis. W tym przykładzie Eureka nie odpytuje samodzielnie endpointu `/health`. Po utracie połączenia z Eureką wcześniej poznane adresy mogą nadal pozwalać na wywołania; uruchomienie nowego klienta bez kopii rejestru to inna sytuacja.

Konfiguracja a odnajdywanie

Adres serwera Eureka jest konfiguracją środowiska, np. `EUREKA_SERVER_URL`. Adres instancji wywoływanej usługi należy natomiast pobrać z rejestru. W endpointzie biznesowym nie należy wpisywać na stałe `http://localhost:8101` dla zależnej usługi.

W przykładzie wykorzystano `py-eureka-client`, czyli zewnętrzną bibliotekę Python do komunikacji z Eureka Server. Nie jest to oficjalny klient Netflix. Eureka udostępnia API REST dla klientów nie-Java; dokumentację protokołu należy porównać z dokumentacją biblioteki.

4. Przygotowanie środowiska

Pobranie materiałów i uruchomienie Eureka Server

Pakiet pomocniczy LAB3 można pobrać jako archiwum ZIP albo korzystać z katalogu lab3-support. Pakiet zawiera lokalny projekt Eureka Server, minimalną parę usług FastAPI oraz test przykładowego wywołania.

Należy rozpakować archiwum obok własnego projektu. Polecenia poniżej są przeznaczone dla PowerShella; katalogiem roboczym terminali jest `lab3-support`, zawierający `requirements.txt`, `compose.yaml` i `demo`.

Plik lub katalog	Zastosowanie
`eureka-server/`	Gotowy serwer rejestru, bez konieczności pisania kodu Java
`compose.yaml`	Uruchomienie wyłącznie rejestru w Dockerze
`demo/shared/eureka.py`	Konfiguracja klienta Eureki
`demo/locker_service/app/main.py`	Udostępnienie danych paczkomatu
`demo/parcel_service/app/main.py`	Wywołanie paczkomatu po nazwie usługi
`demo/tests/test_parcel_service.py`	Testy bez prawdziwego rejestru
`demo/README-LAB3-template.md`	Sekcje do dopisania do dokumentacji projektu

Każda osoba uruchamia rejestr na swoim komputerze. Należy wybrać **jeden** wariant: Docker Compose, jeśli działa lokalny silnik Dockera, albo Maven przy dostępnym JDK 21. Docker nie jest obowiązkowy. W obu wariantach usługi Python należy uruchomić lokalnie jako osobne procesy.

Wymagany jest Python 3.12 lub nowszy. Przed zajęciami należy wybrać sposób uruchomienia Eureki i sprawdzić wymagane narzędzia. W przypadku Dockera należy wykonać:

```
python --version
docker version
docker compose version
```

Polecenie `docker version` powinno wyświetlić sekcję `Server`. Jeśli widoczna jest tylko sekcja klienta albo pojawia się błąd połączenia z silnikiem, należy uruchomić Docker Desktop i ponowić sprawdzenie. Gdy Docker nie może zostać uruchomiony, wariant Maven wymaga JDK 21 i Mavena; ich dostępność należy sprawdzić poleceniami `java -version` i `mvn -version`. Pakiet ZIP nie zawiera środowiska wirtualnego; należy je utworzyć w następnym kroku.

JDK dostarcza narzędzia do kompilacji i uruchamiania kodu Java, a Maven pobiera zależności oraz buduje przygotowany serwer na podstawie `pom.xml`. Wynik `mvn -version` powinien wskazywać Java 21. Jeżeli polecenia nie są dostępne, należy przygotować JDK 21 i Maven przed zajęciami. Samo `pip install` nie instaluje serwera Eureka ani narzędzi Java.

Wariant A — bez Dockera. W osobnym terminalu, rozpoczynając w katalogu `lab3-support`, należy wykonać:

```
Set-Location .\eureka-server
mvn spring-boot:run
```

Przy pierwszym uruchomieniu Maven pobierze zależności. Serwer będzie dostępny pod adresem `http://localhost:8761/`; proces można zatrzymać skrótem `Ctrl+C`. Należy wybrać tę metodę albo Docker Compose.

Wariant B — Docker Compose. Jeśli wybrano Docker, w katalogu `lab3-support` należy wykonać:

```
docker compose up --build -d eureka
```

Budowanie obrazu pobiera zależności Maven i może potrwać przy pierwszym uruchomieniu. Panel Eureka należy otworzyć w przeglądarce. Serwer powinien zostać uruchomiony przed usługami i pozostać aktywny podczas testu.

Przydatne polecenia:

```
docker compose logs -f eureka
docker compose stop eureka
docker compose start eureka
```

Po ćwiczeniu serwer należy zatrzymać wybraną metodą: dla Docker Compose należy wykonać `docker compose down` w katalogu `lab3-support`, a dla Maven użyć skrótu `Ctrl+C` w terminalu z uruchomioną Eureką.

Instalacja zależności przykładu

W katalogu `lab3-support` należy utworzyć środowisko Pythona i zainstalować zależności:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Pakiet pomocniczy przypina `py-eureka-client` do wersji `0.13.3`, aby przykład był odtwarzalny. W projekcie studenta należy dodać tę zależność do `requirements.txt` i używać tej samej wersji podczas laboratorium.

Przed uruchomieniem procesów można wykonać `.\.venv\Scripts\python.exe -m pytest -q`. Testy z paczki korzystają z zamiennika klienta i nie wymagają Javy, Dockera ani działającej Eureki. Nie zastępują jednak sprawdzenia rzeczywistej komunikacji.

Uruchomienie przykładu wspólnego

Przykład pokazuje rejestrację `LOCKER-SERVICE` i `PARCEL-SERVICE`. `PARCEL-SERVICE` wywołuje endpoint paczkomatu, odnajdując `LOCKER-SERVICE` przez Eurekę.

Przykład jest celowo uproszczony: `Locker Service` udostępnia jeden stały rekord o polach zgodnych z LAB1, a `Parcel Service` sprawdza istnienie miejsca docelowego. Nie wykonuje rezerwacji skrytki ani zapisu paczki. Kod demonstracyjny nie zawiera persystencji i cache; we własnym projekcie należy zachować implementację LAB2.

Przed uruchomieniem demo należy zatrzymać własne procesy zajmujące `8101` i `8102` albo zmienić porty demo. W każdym terminalu należy ustawić zmienne osobno. `SERVICE_PORT` to port **zgłaszany do Eureki**, a `--port` to port **otwierany przez Uvicorna** — muszą mieć tę samą wartość. Zmiana zmiennej wymaga ponownego uruchomienia procesu; na czas eksperymentów należy pracować bez `--reload`.

W pierwszym terminalu, w katalogu `lab3-support`, należy uruchomić:

```
$env:EUREKA_SERVER_URL = 'http://127.0.0.1:8761'
$env:SERVICE_NAME = 'LOCKER-SERVICE'
$env:SERVICE_PORT = '8101'
.\.venv\Scripts\python.exe -m uvicorn demo.locker_service.app.main:app --port 8101
```

W drugim terminalu, również w katalogu `lab3-support`, należy uruchomić:

```
$env:EUREKA_SERVER_URL = 'http://127.0.0.1:8761'
$env:SERVICE_NAME = 'PARCEL-SERVICE'
$env:SERVICE_PORT = '8102'
.\.venv\Scripts\python.exe -m uvicorn demo.parcel_service.app.main:app --port 8102
```

Należy poczekać, aż panel Eureka pokaże obie usługi jako `UP`. Następnie w trzecim terminalu należy sprawdzić odpowiedź bezpośrednią i wywołanie przez drugą usługę:

```
Invoke-RestMethod http://127.0.0.1:8101/lockers/KRK-1024
Invoke-RestMethod http://127.0.0.1:8102/delivery-check/KRK-1024
```

Drugie żądanie przechodzi przez `PARCEL-SERVICE`, która wybiera adres `LOCKER-SERVICE` z kopii rejestru i wysyła do niego HTTP. Należy sprawdzić status i treść przez `curl.exe -i http://127.0.0.1:8102/delivery-check/KRK-1024`. Oczekiwane są HTTP `200` oraz JSON:

```
{
  "destination": "KRK-1024",
  "locker": {
    "id": "KRK-1024",
    "name": "Galeria",
    "address": "Dluga 10",
    "city": "Krakow",
    "status": "online"
  }
}
```

Dla `curl.exe -i http://127.0.0.1:8102/delivery-check/UNKNOWN` należy oczekiwać `404` i pola `detail`. Jeśli `curl.exe` nie jest dostępny, można wykonać żądanie w `http://127.0.0.1:8102/docs`. PowerShell może wyświetlić odpowiedź 4xx/5xx jako wyjątek `Invoke-RestMethod`; samo to nie oznacza awarii procesu.

Widoczność wpisu w panelu i dostępność adresu w kopii rejestru klienta mogą nastąpić w różnych chwilach. Jeśli pierwszy odczyt daje `503`, należy sprawdzić `/health` obu usług i logi, odczekać na odświeżenie rejestru i ponowić żądanie.

5. Analiza przykładu przed implementacją

Na podstawie kodu przykładu należy wskazać:

1. Jakie informacje wysyła usługa podczas rejestracji?
2. Po co klient wysyła heartbeat?
3. Który adres jest konfiguracją, a który jest pobierany z rejestru?
4. Czy żądanie biznesowe przechodzi przez Eurekę?
5. Która usługa jest właścicielem informacji o paczkomacie?
6. Co zwraca `PARCEL-SERVICE`, gdy `LOCKER-SERVICE` nie odpowie?
7. Czy dwie instancje `LOCKER-SERVICE` spełniają wymóg dwóch różnych usług?
8. Dlaczego brak paczkomatu powinien dawać inny wynik niż brak połączenia z usługą?

Przykład jest krótki i służy do rozpoznania mechanizmu. Nie trzeba przepisywać go przed rozpoczęciem własnej implementacji.

6. Rejestracja i odnajdywanie usług we własnym projekcie

Krok 1 — weryfikacja wersji z LAB2

Należy uruchomić dotychczasowe testy i potwierdzić działanie pierwszej usługi, bazy oraz cache. Następnie należy utworzyć commit etapu przed zmianami, np.:

```
git status
git add app tests requirements.txt README.md
git diff --cached --stat
git commit -m "chore(lab2): zapisz dzialajacy etap przed LAB3"
```

Jeżeli etap LAB2 jest już zatwierdzony, nowy commit nie jest potrzebny.

Krok 2 — wyznaczenie granicy drugiej usługi

Należy wrócić do podziału domeny i kontraktu z LAB1. Dla obu usług trzeba zapisać w tabeli:

Usługa	Odpowiedzialność	Właściciel danych	Endpoint potrzebny do scenariusza	Port
Pierwsza usługa	…	…	…	dotychczasowy
Druga usługa	…	…	…	wolny port

Należy wybrać taki scenariusz, w którym jedna usługa potrzebuje informacji należącej do drugiej. Nie należy kopiować jej tabel ani bezpośrednio łączyć się z jej bazą.

Przykład decyzji dla własnej domeny: usługa rezerwacji sal pobiera z usługi sal informacje o wskazanej sali. Usługa sal odpowiada za opis sali, a usługa rezerwacji za termin i reguły rezerwacji. Należy zapisać konkretny kontrakt: nazwę usługi, metodę, ścieżkę, wymagane pola JSON i znaczenie błędu. Zdalne sprawdzenie dostępności samo w sobie nie gwarantuje atomowej rezerwacji; do pierwszej integracji warto wybrać odczyt danych opisowych.

Przed kodowaniem własnego scenariusza należy odpowiedzieć: kto potrzebuje tych danych, dlaczego należą do drugiej usługi oraz co powinien otrzymać odbiorca, gdy zależność nie odpowiada. Wynikiem kroku jest tabela usług i jeden uzasadniony kontrakt, a nie nowy temat projektu.

Krok 3 — przygotowanie drugiej usługi

Należy utworzyć osobną aplikację FastAPI z własnym `create_app()`, własną nazwą, portem i co najmniej jednym endpointem domenowym. Na potrzeby etapu endpoint może korzystać z małego lokalnego zestawu danych, jeżeli persystencja drugiej usługi nie jest jeszcze zaimplementowana. Dane pierwszej usługi pozostają w jej bazie.

Obie aplikacje należy uruchamiać jako odrębne procesy. W konfiguracji każdej należy ustawić:

- nazwę logiczną usługi,
- port nasłuchiwania,
- adres serwera Eureka,
- adres IP instancji osiągalny dla drugiej usługi.

Lokalnie można użyć adresu `127.0.0.1`; adresu zarejestrowanego w Eurece nie należy zgadywać, jeżeli usługi są uruchamiane w innym środowisku.

Zmienna	Przykład dla pierwszej usługi	Znaczenie
`SERVICE_NAME`	`ROOM-SERVICE`	Nazwa, pod którą druga usługa będzie szukała zależności
`SERVICE_PORT`	`8101`	Port zgłaszany do rejestru, zgodny z `uvicorn --port`
`INSTANCE_IP`	`127.0.0.1`	Adres osiągalny z procesu wywołującego
`INSTANCE_HOST`	`localhost`	Nazwa hosta w zgłoszeniu; domyślnie klient używa jej do połączenia
`EUREKA_SERVER_URL`	`http://127.0.0.1:8761`	Adres rejestru, bez końcowego `/eureka`

`0.0.0.0` oznacza nasłuchiwanie na interfejsach i nie jest adresem docelowym do zgłoszenia w Eurece. Dla układu z instrukcji należy zachować `127.0.0.1`; wewnątrz VM lub kontenera `localhost` wskazuje to środowisko. Nie należy przenosić pojedynczego procesu do VM bez zmiany adresów osiągalnych dla pozostałych procesów.

W bibliotece wywołanie `do_service()` domyślnie korzysta z nazwy hosta instancji. Dlatego `INSTANCE_HOST` i `INSTANCE_IP` powinny wskazywać ten sam osiągalny komputer; ustawienie samego IP nie poprawi błędnej nazwy hosta.

Należy dodać `py-eureka-client==0.13.3` do istniejącego `requirements.txt` każdej usługi oraz ponownie zainstalować zależności jej środowiska. Nie należy zastępować pliku zależności LAB2 plikiem z demo.

Krok 4 — rejestracja cyklu życia aplikacji

Należy utworzyć klienta Eureka i uruchamiać go w cyklu życia FastAPI. Poniższy szkielet pokazuje wariant dla drugiej usługi, która nie ma jeszcze zasobów LAB2. Endpointy należy zarejestrować przed `return app`. Dla pierwszej usługi należy zastosować połączenie cykli życia opisane pod szkieletem.

```python
import os
from contextlib import asynccontextmanager
from fastapi import FastAPI
from py_eureka_client.eureka_client import EurekaClient


def create_app(eureka_client: EurekaClient | None = None) -> FastAPI:
    client = eureka_client if eureka_client is not None else EurekaClient(
        eureka_server=os.getenv("EUREKA_SERVER_URL", "http://127.0.0.1:8761"),
        eureka_context="/eureka",
        app_name=os.getenv("SERVICE_NAME", "PARCEL-SERVICE"),
        instance_ip=os.getenv("INSTANCE_IP", "127.0.0.1"),
        instance_host=os.getenv("INSTANCE_HOST", "localhost"),
        instance_port=int(os.getenv("SERVICE_PORT", "8102")),
        strict_service_error_policy=False,
    )

    @asynccontextmanager
    async def lifespan(app: FastAPI):
        app.state.eureka_client = client
        try:
            await client.start()
            yield
        finally:
            await client.stop()

    app = FastAPI(lifespan=lifespan)
    # Tutaj należy zarejestrować endpointy własnej usługi.
    return app


app = create_app()
```

Nazwę, URL rejestru, IP i port należy pobierać z konfiguracji środowiska zamiast utrwalać w kodzie produkcyjnym. Dla przygotowanego serwera Spring Cloud należy podać adres bazowy, np. `http://127.0.0.1:8761`, a ścieżkę API ustawić jako `eureka_context="/eureka"`. W każdej usłudze klient należy utworzyć wewnątrz `create_app()`, aby zachować wzorzec projektu z LAB1 i ułatwić izolowanie aplikacji w testach. Fragment pokazuje kolejność: utworzenie klienta, start przy uruchamianiu aplikacji i stop przy jej zamykaniu.

`async def` definiuje funkcję asynchroniczną, a `await` służy tutaj do oczekiwania na operację klienta. `app.state` przechowuje zasób dostępny dla endpointów tej aplikacji. Kod przed `yield` wykonuje przygotowanie, a `finally` — sprzątanie przy zakończeniu. Klienta należy uruchomić raz na instancję aplikacji, a nie tworzyć przy każdym żądaniu.

Połączenie z LAB2: w istniejącym `create_app(database_url=None, redis_client=None, ...)` należy zachować parametry bazy i Redis oraz dodać `eureka_client=None`. Dotychczasową funkcję `lifespan`, tworzącą bazę i cache, należy nazwać `storage_lifespan`. Po utworzeniu klienta Eureki w tej samej fabryce należy dodać:

```python
@asynccontextmanager
async def lifespan(app: FastAPI):
    async with storage_lifespan(app):
        app.state.eureka_client = client
        try:
            await client.start()
            yield
        finally:
            await client.stop()
```

`storage_lifespan` oznacza istniejącą funkcję z projektu, a nie import z biblioteki. `FastAPI(lifespan=lifespan)` powinno korzystać z nowej funkcji. Należy zachować dotychczasową rejestrację endpointów. Kolejność pracy wynosi: przygotowanie bazy i cache → start klienta Eureki → obsługa żądań → zatrzymanie klienta → zamknięcie zasobów LAB2. Zastąpienie całej fabryki pustym szkieletem usunęłoby istniejące API i inicjalizację bazy.

Parametr `strict_service_error_policy=False` jest potrzebny do zachowania mapowania błędów z Kroku 5. Przy wartości domyślnej `True` klient może potraktować odpowiedź HTTP jako błąd instancji, spróbować kolejnego węzła i po niepowodzeniu zgłosić `WalkNodeException`, która jest podklasą `URLError`. Wtedy odpowiedź `404` z usługi docelowej mogłaby zostać zamieniona na `503`. Ustawienie `False` pozwala przekazać `HTTPError` do kodu endpointu, gdzie można odwzorować `404` i pozostałe błędy HTTP.

Każda usługa powinna zarejestrować własną nazwę i port. Należy potwierdzić w panelu Eureki, że obie instancje mają stan `UP`.

Krok 5 — wykonanie żądania przez nazwę usługi

W wybranym endpointzie `async def` należy pobrać dane z drugiej usługi przez klienta Eureka. Poniższy fragment działa wewnątrz funkcji endpointu, gdzie `app` pochodzi z otaczającego `create_app()`:

```python
result = await app.state.eureka_client.do_service(
    "LOCKER-SERVICE",
    "/lockers/KRK-1024",
    return_type="json",
    timeout=3,
)
```

Należy zastąpić nazwę i ścieżkę własnym kontraktem. Nie należy wpisywać adresu `http://127.0.0.1:8101` w kodzie endpointu. Adres Eureki pozostaje parametrem konfiguracyjnym, a adres wywoływanej usługi pochodzi z rejestru.

Należy wykorzystać identyfikator przekazany do endpointu, np. `f"/lockers/{locker_id}"`, zamiast pozostawić stałe `KRK-1024`. `return_type="json"` zleca zdekodowanie odpowiedzi, a `timeout=3` ogranicza oczekiwanie pojedynczego wywołania HTTP. Nie jest to gwarancja czasu całego przepływu, zwłaszcza przy kilku instancjach. Lokalny endpoint wykorzystujący synchroniczne SQLAlchemy może pozostać `def`; nie należy przenosić blokujących operacji bazy do `async def` bez przemyślenia sposobu ich wykonania.

Biblioteka `py-eureka-client` może zgłosić `urllib.error.HTTPError`, gdy usługa zwróci błąd HTTP, albo `urllib.error.URLError`, gdy nie można jej odnaleźć lub nawiązać połączenia. Należy przełożyć te wyjątki na kontrolowaną odpowiedź FastAPI. Przykład mapuje `404` usługi docelowej na `404`, pozostałe błędy HTTP na `502`, a błąd odnalezienia lub połączenia na `503`:

```python
from urllib.error import HTTPError, URLError
from fastapi import HTTPException

try:
    result = await app.state.eureka_client.do_service(
        "LOCKER-SERVICE",
        "/lockers/KRK-1024",
        return_type="json",
        timeout=3,
    )
except HTTPError as exc:
    if exc.code == 404:
        raise HTTPException(
            status_code=404, detail="Nie znaleziono zasobu"
        ) from exc
    raise HTTPException(
        status_code=502, detail="Zależna usługa zwróciła błąd HTTP"
    ) from exc
except (URLError, TimeoutError) as exc:
    raise HTTPException(
        status_code=503, detail="Zależna usługa jest niedostępna"
    ) from exc
```

Odpowiedź nie powinna ujawniać tracebacku ani sekretów. Nie należy dodawać na tym etapie pełnego circuit breakera ani mechanizmu ponawiania żądań.

Sytuacja w przykładzie	Odpowiedź `PARCEL-SERVICE`	Znaczenie
Paczkomat istnieje	`200` z danymi	Wywołanie zakończone poprawnie
`LOCKER-SERVICE` zwróciła `404`	`404`	Brak wskazanego paczkomatu
`LOCKER-SERVICE` zwróciła inny błąd HTTP, np. `500`	`502`	Zależność odpowiedziała błędem
Brak instancji, brak połączenia lub timeout	`503`	Nie udało się uzyskać odpowiedzi zależności

`HTTPError` dziedziczy po `URLError`, więc należy obsłużyć go wcześniej. Powyższe mapowanie dotyczy przykładu odczytu. We własnym kontrakcie należy jawnie określić znaczenie błędów, w tym ewentualnego konfliktu `409`. Nie należy zwracać pozornego `200` z pustymi danymi po awarii zależności.

Krok 6 — sprawdzenie odporności na błąd zależności

Należy wykonać poniższe scenariusze, dostosowując ścieżki i identyfikatory do projektu:

Scenariusz	Działanie	Oczekiwany rezultat
Poprawny odczyt	Obie usługi działają; wywołanie endpointu zależnego	`200` i dane właściwego zasobu
Brak zasobu	Wywołanie z nieistniejącym ID przy działającej zależności	`404`, bez zatrzymania procesów
Niedostępna zależność	Zatrzymanie tylko usługi wywoływanej przez Ctrl+C; ponowienie żądania	`503`; `/health` usługi wywołującej nadal daje `200`
Przywrócenie zależności	Ponowne uruchomienie zatrzymanej usługi i odświeżenie rejestru klienta	Ponownie `200` z danymi

W demo należy zatrzymać terminal `LOCKER-SERVICE`, pozostawiając Eurekę i `PARCEL-SERVICE`. Status i treść odpowiedzi należy odczytać przez `curl.exe -i http://127.0.0.1:8102/delivery-check/KRK-1024`. Dodatkowo należy sprawdzić `curl.exe -i http://127.0.0.1:8102/health`. Dla każdego scenariusza należy zapisać stan początkowy, żądanie, wynik i sposób przywrócenia środowiska.

Przy Ctrl+C klient próbuje się wyrejestrować; przy nagłej awarii takiej możliwości nie ma. Wpis może przez pewien czas pozostawać w rejestrze lub jego lokalnej kopii. Eureka stosuje również mechanizm self-preservation, który przy dużym spadku liczby heartbeatów może wstrzymać usuwanie wpisów. Nie należy oczekiwać usunięcia dokładnie po zadanej liczbie sekund ani wyłączać tego mechanizmu, aby uzyskać oczekiwany wynik ćwiczenia. Należy opisać czas obserwacji oraz faktyczny rezultat żądania.

Po teście należy ponownie uruchomić usługę i potwierdzić, że przepływ działa.

Eksperyment dodatkowy: można zmienić port usługi wywoływanej jednocześnie w `SERVICE_PORT` i `--port`, pozostawiając jej nazwę logiczną. Po odświeżeniu rejestru należy wykazać, że klient odnajduje nowy adres bez zmiany swojego kodu. Samo zatrzymanie Eureki nie jest równoważne zatrzymaniu usługi — wywołania mogą korzystać z wcześniej pobranych adresów.

Krok 7 — testy automatyczne

Należy dodać co najmniej cztery własne testy: poprawny odczyt i użycie nazwy logicznej, brak zasobu (`404`), niedostępność zależności (`503`) oraz inny błąd HTTP zależności (`502` w przykładzie). Należy sprawdzać kod i istotne pola odpowiedzi. Jeżeli własny kontrakt wymaga innego mapowania, trzeba je opisać i przetestować.

Testy nie powinny wymagać działającego Eureka Server. Należy wstrzyknąć zamiennik klienta, który udostępnia asynchroniczne metody `start()`, `stop()` i `do_service()`. Poniższy przykład sprawdza wywołanie usługi po nazwie. Ścieżkę importu i endpoint należy dostosować do własnego projektu:

```python
import pytest
from httpx import ASGITransport, AsyncClient

from app.main import create_app


class FakeEurekaClient:
    def __init__(self, response=None, error=None):
        self.response = response if response is not None else {"id": "KRK-1024", "status": "online"}
        self.error = error
        self.calls = []

    async def start(self):
        pass

    async def stop(self):
        pass

    async def do_service(self, service_name, path, return_type=None, timeout=None):
        self.calls.append((service_name, path, return_type, timeout))
        if self.error:
            raise self.error
        return self.response


@pytest.fixture
def anyio_backend():
    return "asyncio"


@pytest.mark.anyio
async def test_endpoint_uses_service_discovery():
    fake = FakeEurekaClient()
    app = create_app(eureka_client=fake)

    async with app.router.lifespan_context(app):
        async with AsyncClient(
            transport=ASGITransport(app=app), base_url="http://test"
        ) as client:
            response = await client.get("/delivery-check/KRK-1024")

    assert response.status_code == 200
    assert response.json()["locker"]["id"] == "KRK-1024"
    assert fake.calls == [
        ("LOCKER-SERVICE", "/lockers/KRK-1024", "json", 3)
    ]
```

Drugi test powinien wstrzyknąć atrapę zgłaszającą `URLError` i sprawdzić kontrolowaną odpowiedź `503`. Testy powinny również sprawdzać odpowiedź dla błędu HTTP zwróconego przez zależną usługę.

`ASGITransport` pozwala wywołać aplikację bez Uvicorna i portu TCP, ale sam nie uruchamia jej `lifespan`; dlatego przykład jawnie otwiera `app.router.lifespan_context(app)`. Fixture `anyio_backend` wybiera `asyncio`. Należy zachować testy LAB1 i LAB2: w testach pierwszej usługi trzeba podać jednocześnie bazę w `tmp_path`, atrapę Redis i atrapę Eureki. Dodanie rejestracji nie powinno powodować rzeczywistych połączeń sieciowych w starych testach.

Testy powinny weryfikować:

- kod i treść odpowiedzi endpointu wywołującego,
- nazwę logiczną oraz ścieżkę przekazane do klienta,
- kontrolowaną odpowiedź przy błędzie zależności.

Test rzeczywistej rejestracji i odnajdywania należy wykonać ręcznie z uruchomioną Eureką. Nie należy wykonywać połączeń sieciowych w podstawowym zestawie testów `pytest`.

Pełniejszy przykład testów z zamiennikiem znajduje się w `lab3-support/demo/tests/test_parcel_service.py`. Należy też sprawdzić, że start i stop klienta są wywołane przy otwarciu i zamknięciu aplikacji. Atrapa potwierdza sposób użycia klienta; nie dowodzi poprawności adresu w prawdziwym rejestrze.

Krok 8 — aktualizacja dokumentacji

Do README własnego projektu należy dopisać sekcje z szablonu dokumentacji LAB3. Dokumentacja powinna opisywać oba serwisy, ich właścicieli danych, porty, nazwy rejestrowane w Eurece, konfigurację, endpoint komunikacji i rzeczywiste wyniki testów.

7. Diagnostyka konfiguracji i granice zadania

Rozpoznawanie problemów z uruchomieniem

Objaw	Co należy sprawdzić
Panel `8761` nie odpowiada	Proces Maven albo `docker compose logs eureka`; rejestr musi być uruchomiony przed usługami
`ModuleNotFoundError` dla `demo`	Katalog roboczy powinien zawierać `demo` i `requirements.txt`; należy używać Pythona z właściwego `.venv`
Port jest zajęty	`Get-NetTCPConnection -LocalPort 8101 -State Listen`; należy ustalić, czy działa własna wcześniejsza usługa
`/health` działa, ale brak wpisu w rejestrze	Logi klienta, `EUREKA_SERVER_URL`, `eureka_context` oraz uruchomienie `lifespan`
Wpis jest `UP`, ale wywołanie daje `503`	Zgodność `SERVICE_PORT` z `--port`, osiągalność `INSTANCE_HOST` i `INSTANCE_IP`, nazwę usługi i aktualność lokalnej kopii rejestru
Odpowiedź `404` mimo wpisu `UP`	Ścieżkę endpointu i istnienie zasobu; wpis w rejestrze dotyczy instancji, nie paczkomatu
Stare testy próbują łączyć się z Eureką	Przekazanie atrapy klienta do każdej testowanej fabryki aplikacji

Po usunięciu przyczyny należy ponownie uruchomić zmieniony proces i sprawdzić ten sam scenariusz. Nie należy kończyć przypadkowych procesów tylko dlatego, że zajmują port.

Zakres implementacji

W tym etapie nie należy dodawać API Gateway, RabbitMQ, komunikacji asynchronicznej, Sagi, klastra Eureki, własnych obrazów kontenerowych ani pełnego Docker Compose dla wszystkich usług. Docker Compose z pakietu służy wyłącznie do uruchomienia przygotowanego rejestru. Nie należy zmieniać modelu persystencji ani cache z LAB2, jeśli nie jest to konieczne do zachowania działania istniejącego API.

8. Punkty kontrolne i kryteria zaliczenia etapu

Punkty kontrolne

Punkt	Oczekiwany rezultat
A — ciągłość	Testy z LAB2 przechodzą, działający etap został zachowany w Git
B — granice usług	Dwie usługi mają różne odpowiedzialności, osobne porty i właścicieli danych
C — rejestracja	Eureka pokazuje obie instancje jako `UP`
D — odnajdywanie	Jedna usługa wywołuje drugą przez nazwę logiczną pobraną z rejestru
E — błąd	Niedostępność zależnej usługi daje kontrolowaną odpowiedź i została opisana
F — weryfikacja	Testy jednostkowe przechodzą, test ręczny integracji został wykonany

Podczas odbioru należy własnymi słowami wyjaśnić:

1. Który komponent zna adres Eureki, a który adres zależnej usługi?
2. Dlaczego żądanie dotyczące paczkomatu nie przechodzi przez serwer Eureka?
3. Dlaczego `UP`, odpowiedź `/health` i poprawna odpowiedź domenowa to różne dowody?
4. Czym różnią się awaria rejestru, awaria usługi i brak zasobu?
5. Gdzie kończy się odpowiedzialność każdej z dwóch usług?

Każda osoba powinna umieć wykonać małą zmianę, np. dodać test nowego identyfikatora albo zmienić port i wykazać odnalezienie instancji. W przypadku użycia AI należy opisać zakres pomocy i sposób sprawdzenia kodu; dopuszczalne jest wsparcie implementacji, ale decyzje domenowe i wyniki testów trzeba umieć uzasadnić.

Kryteria oceny

Obszar	Element	Punkty
Kod	Zachowanie pierwszej usługi z LAB2: API, baza, cache i testy	10
Kod	Druga odrębna usługa z własną odpowiedzialnością i endpointem domenowym	15
Kod	Rejestracja obu usług, konfiguracja adresów i poprawny cykl życia klienta	15
Kod	Wywołanie przez nazwę usługi oraz kontrola błędów zależności	10
Kod	Testy izolowane od prawdziwego rejestru	10
Dokumentacja	Granice usług, własność danych, kontrakt i konfiguracja	10
Dokumentacja	Kroki uruchomienia, testu i resetu środowiska	10
Dokumentacja	Wyniki awarii, ograniczenia i wykorzystana dokumentacja	10
Dokumentacja	Wkład autorów i wykorzystanie AI	10
Razem	**Kod 60 + dokumentacja 40**	**100**

Zaliczenie etapu: minimum 50 punktów oraz spełnienie wszystkich warunków minimum:

- pierwsza usługa nadal przechowuje i udostępnia dane zgodnie z etapem LAB2,
- druga usługa ma odrębny endpoint domenowy i nie zapisuje bezpośrednio do bazy pierwszej,
- obie usługi rejestrują się w Eurece pod różnymi nazwami i portami,
- co najmniej jedno żądanie HTTP odnajduje zależność przez nazwę logiczną,
- błąd zależności jest obsłużony kontrolowanym kodem HTTP,
- przechodzą co najmniej cztery własne testy z kroku 7 i wcześniejsze testy projektu, a ręczne scenariusze z kroku 6 są udokumentowane,
- każda osoba potrafi wyjaśnić rejestrację, odnajdywanie i granice obu usług.

9. Oznaczenie i przekazanie wersji projektu LAB3

Należy oddać to samo repozytorium rozwijane w LAB1 i LAB2. Przed dodaniem plików do Git należy sprawdzić, czy nie zawierają sekretów, lokalnych baz, środowiska wirtualnego ani cache Pythona.

```
git status
git add .
git diff --cached --stat
git commit -m "feat(lab3): rejestracja i odnajdywanie usług przez Eurekę"
git tag lab3
git push origin HEAD
git push origin refs/tags/lab3
```

Jeżeli tag `lab3` już istnieje, nie należy nadpisywać go siłą. Należy uzgodnić oznaczenie poprawionej wersji. Nie należy wysyłać lokalnych konfiguracji, haseł ani plików `.env`.

10. Dokumentacja techniczna

W README należy podać wersje użytych narzędzi oraz dwa wykorzystane źródła: dokumentację biblioteki i dokumentację protokołu lub serwera. W przypadku informacji o cyklu życia FastAPI należy wskazać wykorzystane źródło dotyczące lifespan.

Dla każdego wykorzystanego źródła należy wskazać konkretny problem, zastosowany fragment rozwiązania oraz test lub obserwację potwierdzającą rezultat. Sam wykaz linków nie wystarcza.

Prefix API zależy od implementacji serwera. Dokumentacja REST Netflix pokazuje ścieżkę `/eureka/v2`; przygotowany serwer Spring Cloud udostępnia endpointy pod `/eureka/*`. W kliencie przykładowym należy ustawić `eureka_context="/eureka"` i jako `EUREKA_SERVER_URL` podać adres bazowy serwera.

- Eureka REST operations — Netflix — rejestracja, heartbeat, odnajdywanie i wyrejestrowanie klienta nie-Java.
- Self Preservation — Netflix — przyczyny utrzymywania wpisów po utracie heartbeatów.
- Eureka Server — Spring Cloud Netflix — konfiguracja serwera i endpointów rejestru.
- Service Registration and Discovery — Spring — przykład rejestru i dwóch usług komunikujących się po nazwie.
- py-eureka-client — dokumentacja projektu oraz wydanie na PyPI — integracja klienta Python, w tym API asynchroniczne.
- Lifespan Events — FastAPI — uruchamianie i zamykanie klienta razem z aplikacją.
- Async Tests — FastAPI — `ASGITransport`, `AsyncClient` i odrębne uruchomienie cyklu życia w testach.

Nie należy traktować materiałów biblioteki jako źródła oficjalnej specyfikacji Netflix. Przed użyciem kodu z dokumentacji należy sprawdzić zgodność wersji i uruchomić testy.